

Contents

1 - Tokenized bond — register + issue (T0/T1) 2

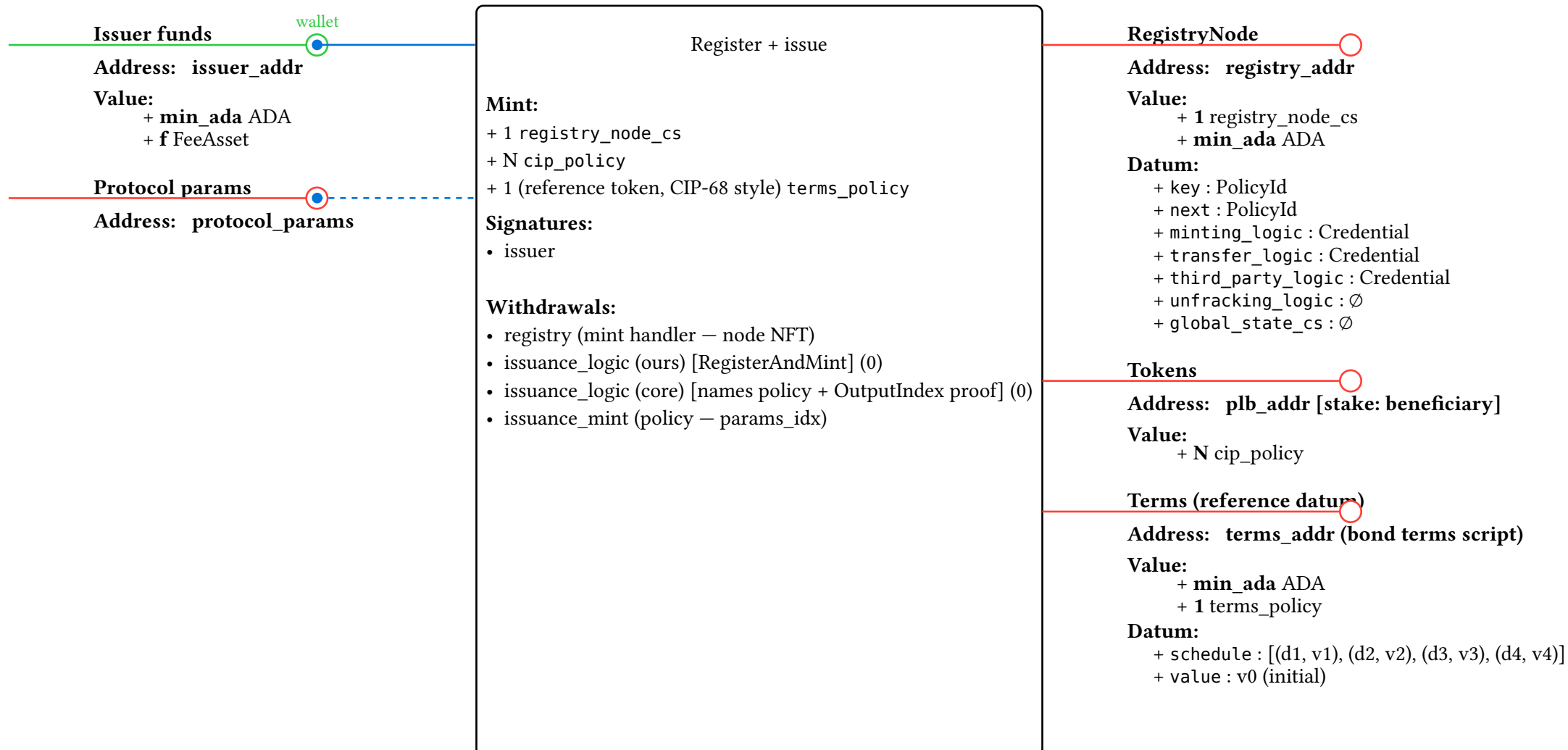
2 - Tokenized bond — free transfer (T2) 3

3 - Tokenized bond — scheduled transformation (T3) 4

4 - Tokenized bond — deadline graduation (T4) 5

1 - Tokenized bond — register + issue (T0/T1)

One transaction creates the CIP-113 registry node **and** mints the first batch to the beneficiary, publishing the instrument’s terms as a CIP-68-style reference datum. The atomic shape is enabled by the core `issuance_logic`’s `OutputIndex` proof — the node is an `OUTPUT` of this tx, not a reference input, so no delegate sees it and custody falls to `no_escape` locally. The strict register-then-mint split is the substandard’s own rule, not a framework limit; this is the deliberate atomic alternative.



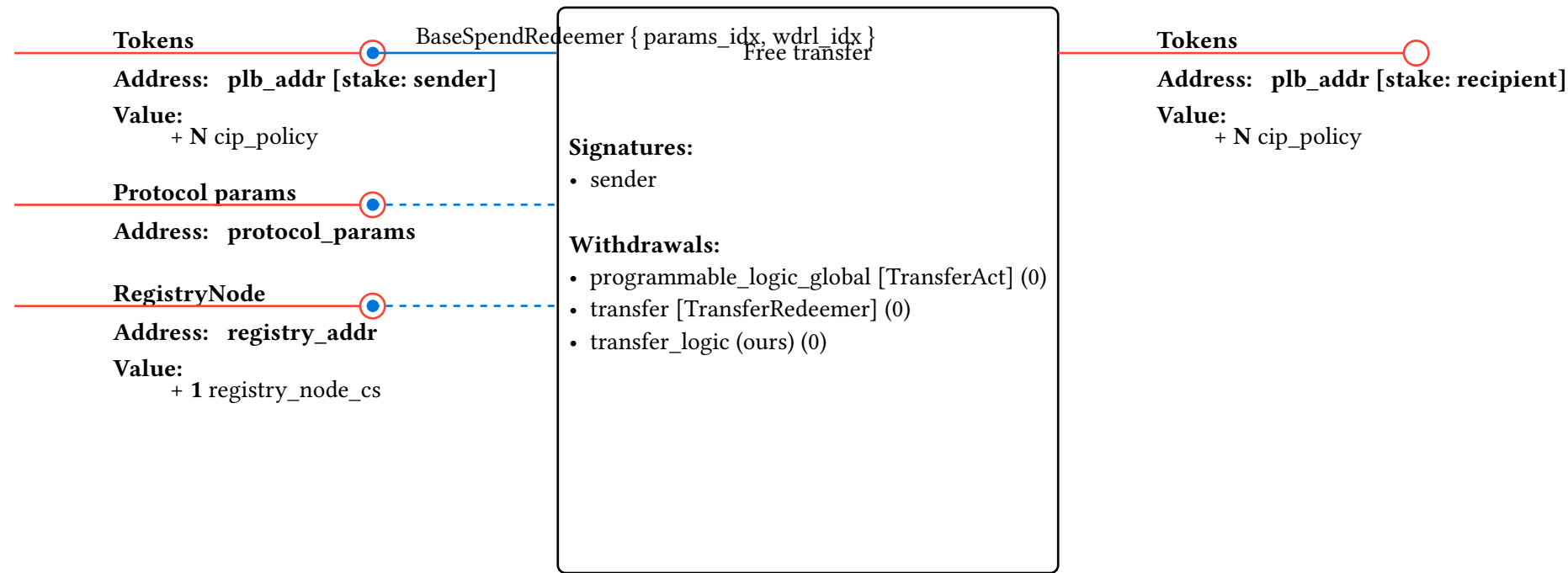
Notes:

- The crux is the proof shape: the core `issuance_logic` carries an `OutputIndex { index }` proof pointing at the **RegistryNode output being created**, not a `RefInput`. In that branch no delegate covers the mint, so custody is enforced by `no_escape` right here — exactly the framework’s “first mint” case. There is no `RegistryNode` reference input: it does not exist yet.
- Our issuance arm is a single `RegisterAndMint` that validates BOTH concerns together — registration authority (issuer signature) + node shape, and the first-batch mint. A single arm is fine “just make that a deliberate choice, not an accident” (09-DEVELOPING-SUBSTANDARDS §7); it must cross-check the tx shape (node NFT minted → node created; `cip_policy` entries present in `tx.mint`) so a caller cannot reuse it in the wrong context.
- Base-layer carve-out makes this safe: the guard “a tx that spends a registry node may not mint/burn that node’s own token” is scoped to the node being **spent** (the covering predecessor — a different policy), never the node being **created**, so minting the new key alongside registration is allowed (09-DEVELOPING-SUBSTANDARDS §7).
- A mint carries **two** issuance withdraw-0s plus the policy: ours (the mode-aware validator) and the protocol’s `issuance_logic` (redeemer is a map naming this policy, here backed by the `OutputIndex` proof). Registration proof of instance: the registry’s mint handler requires our issuance-logic withdraw-0, binding the node’s `minting_logic` to the `issuance_mint` template whose hash must equal key — frozen for the node’s life (09-DEVELOPING-SUBSTANDARDS §3, §7). The same `RegisterAndMint` withdrawal satisfies it.
- Registry registration pins the frozen stance (Q-RULE-2/3): permissive transfer logic, governed-extraction third-party logic, unfracking hook unset (unfracking forbidden), no global state. The substandard reference validators are used as-is. Every substandard script must carry a publish handler accepting `RegisterCredential`, or it can never be registered (09-DEVELOPING-SUBSTANDARDS §4).
- Minted tokens must land at a PLB output with an inline stake credential and a bounded inline datum — enforced by the protocol’s `issuance_logic` (`no_escape`); the substandard does not re-check it. Supply is issuer-gated: single asset name, no per-holder state to mint against.
- The reference token is minted under the **terms policy** (the bond’s own script), deliberately **not** under the governed policy: every governed token is PLB-custodied, and the third-party path preserves datums byte-for-byte — a reference datum under the governed policy could never be updated without the holder’s signature. Companion-asset protection is a substandard decision (the framework has none): a distinct policy id is exactly that decision.
- The schedule is baked as validator constants — fixed 4% annual over four years, precomputed off-chain with a fixed-point scale: [(d1, v1), (d2, v2), (d3, v3), (d4, v4)] with $v4 \approx 1.1699 \times \text{scale}$. No on-chain compounding: the validator looks the current value up by time. The datum is the CIP-68 display mirror; the graduation math reads the baked v4. The CIP-113 tokens themselves carry no datum — the instrument’s condition lives in time (the validity range) and in the terms constants, not in state.
- Trade-off vs the strict split: saves a transaction, a node-creation round-trip, and the register→mint window, at the cost of one arm handling two contexts and the `OutputIndex` proof path. Register-only bootstrapping (global-state init before any token) is moot here — `global_state_cs`: $\emptyset$.

Figure 1: Atomic register + first issue

2 - Tokenized bond — free transfer (T2)

Before and after the deadlines the owner transfers freely (P1). Conversion is never a freeze: an unconverted token keeps moving the same way.



- Notes:**
- Dispatch chain: PLB requires the dispatcher → dispatcher (TransferAct) requires transfer → transfer resolves the registry node and requires our transfer logic (09-DEVELOPING-SUBSTANDARDS §4, §6). A transfer needs **three** script withdrawals, not two — without the dispatcher the tx fails at programmable_logic_base before the transfer validator ever runs.
 - The PLB spend redeemer is BaseSpendRedeemer { params_idx, wdr_l_idx } — one record for every PLB input in the tx; it carries **no action arm**. The path is selected by the dispatcher's redeemer (TransferAct), never by the base redeemer.
 - Withdrawals are listed here by role; the ledger presents them in its canonical order (scripts before vkeys, ascending) — builders derive every wdr_l_idx from the sorted set (09-DEVELOPING-SUBSTANDARDS §10).
 - P1 at every point of the lifecycle — before and after the deadlines: the permissive transfer_logic (Q-RULE-3) never gates who may hold or send, so the token is DEX/venue-compatible.
 - Ownership rides the inline stake credential of the token UTxO — it moves with the token at every transfer, and it is what the graduation binds the payout to (T4).
 - No burn, no issuance, no rule withdraw-0: a transfer never touches the instrument's condition — the schedule lives in time and constants, not in any state.

Figure 2: Free transfer before or after the deadlines

3 - Tokenized bond — scheduled transformation (T3)

At each deadline the instrument’s value inflates 4%: a transaction updates the terms datum to the schedule’s current value. The holder does nothing — the issuer normally submits it, and anyone (including the holder) can force it, because no signature is required.

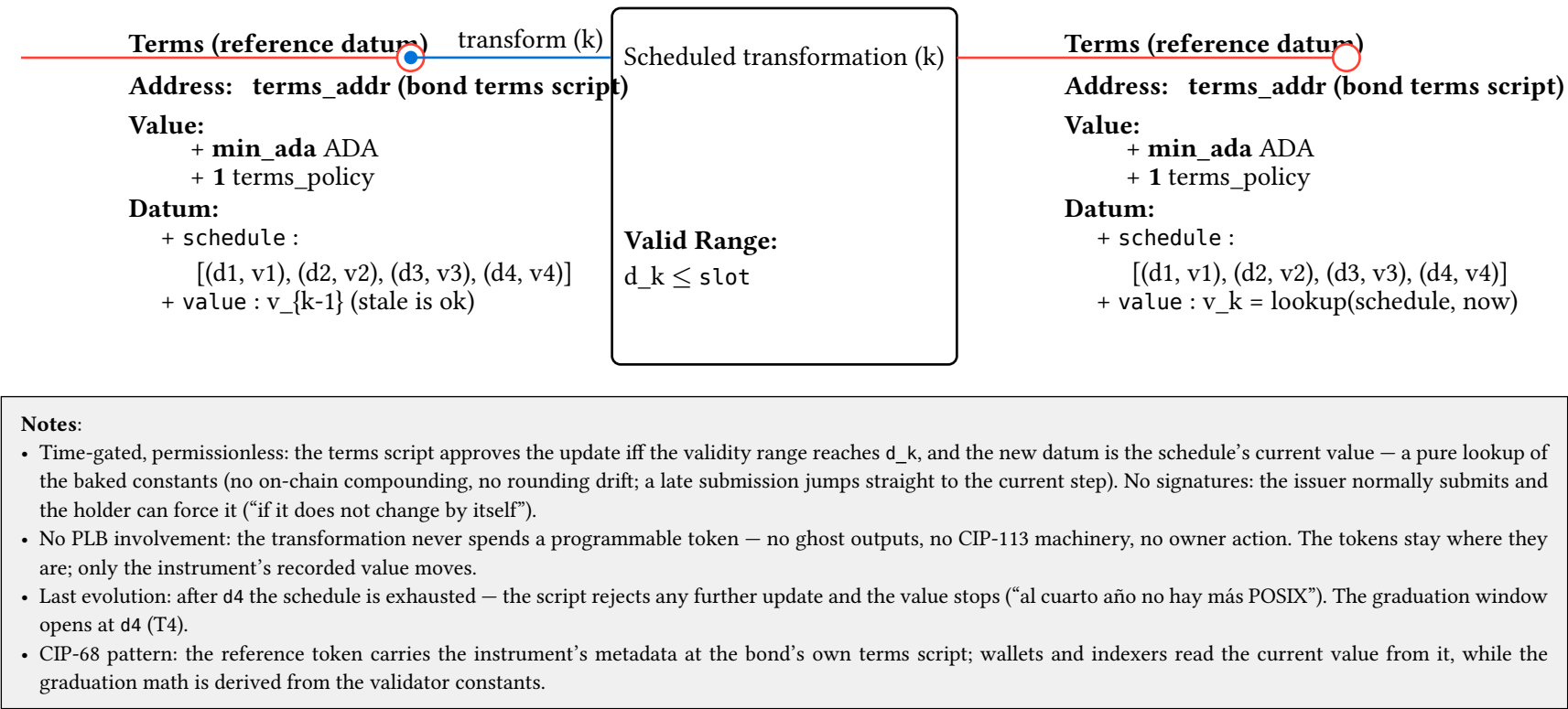
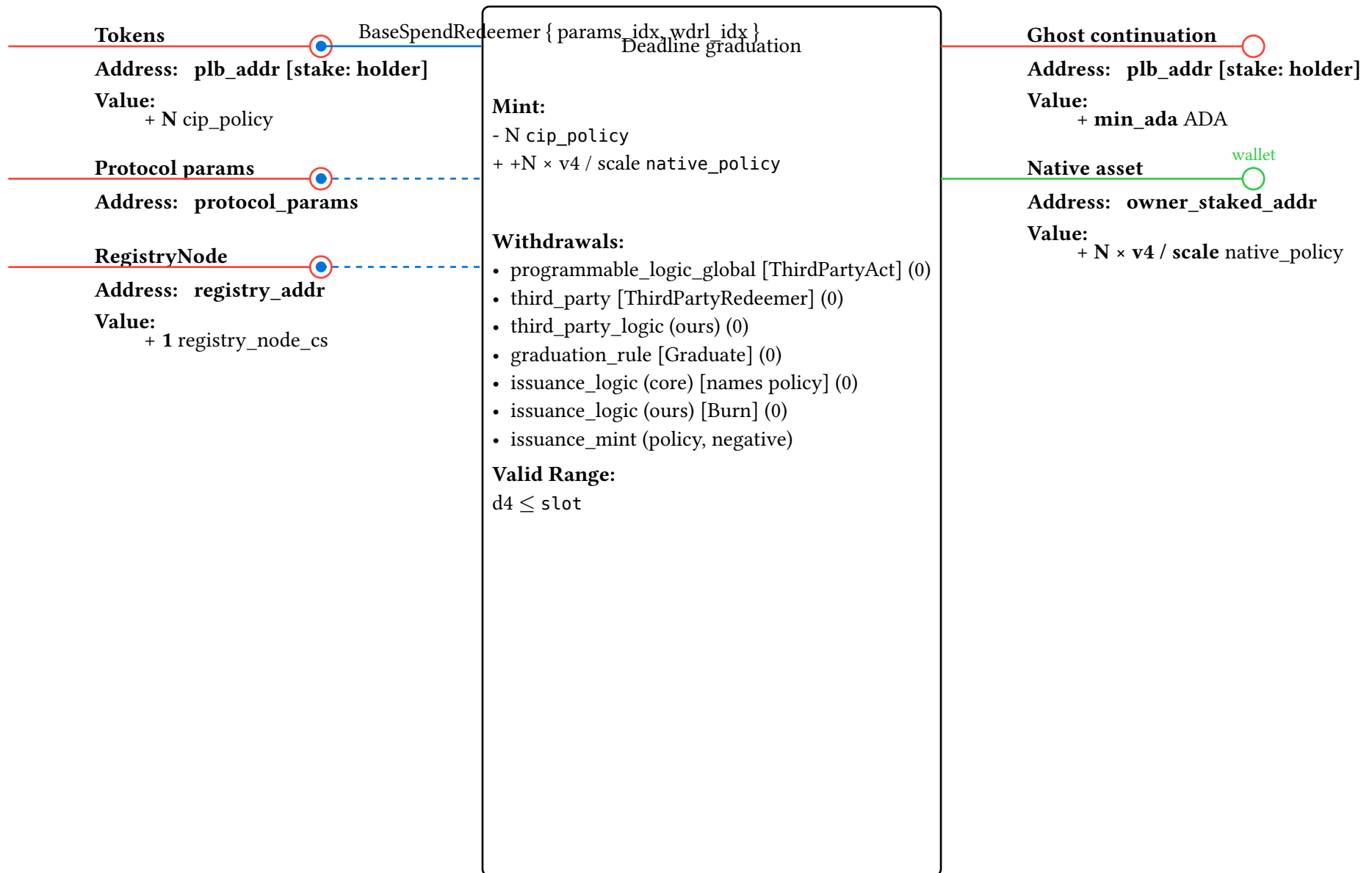


Figure 3: Scheduled value transformation (+4%, holder-passive)

4 - Tokenized bond — deadline graduation (T4)

From $d4$ on the tokens convert into the corresponding native asset at the schedule's final value. Two forms, one function: the issuer drives the **third-party path** (no owner action — a ghost continuation remains) and the owner can drive the **transfer path** (owner-signed — nothing remains). Either way the burn is event-gated and the natives are destination-bound, so there is no attack or benefit in waiting.



Notes:

- Dispatch chain: PLB requires the dispatcher → dispatcher (ThirdPartyAct) requires third_party → third_party resolves the registry node and requires our third-party logic. The third-party path is selected by the **dispatcher's** redeemer (ThirdPartyAct) — the base spend redeemer carries no action arm — and does not require the token owner's signature (09-DEVELOPING-SUBSTANDARDS §6).
- A burn fires issuance_mint with a negative quantity — the **same two** issuance withdraw-0s as a mint (protocol issuance_logic + ours) — plus the full spend chain of whichever action releases the tokens (09-DEVELOPING-SUBSTANDARDS §6).
- Event gate (Q-RULE-1): the rule withdraw-0 (Graduate) approves iff the validity range reaches $d4$ — time-driven, no oracle; the event condition is the tx's own validity range.
- Authorization (Q-GRAD-2, this instrument): the burn requires the authority of **either** the third-party logic (its withdraw-0 — the issuer-assembled path) **or** the owner's signature (the owner path); the event gate ($d4$) applies to both. The third-party path needs no owner action; the owner path needs nothing but the owner.
- The conversion function (the Burn-mode validation) is shared by both paths: burn shape, full-burn per asset name (whole holdings burn — it makes the per-owner destination attribution exact), destination binding and the scaled native mirror. The native minting policy is signer-agnostic: it approves the mint only against the burn ($\text{mint} == \text{burned} \times v4 / \text{scale}$), whichever credential signed it.
- Destination binding: the natives must land in outputs whose address carries the burned token's inline stake credential — the owner's current credential, which moves with the token at every transfer. No owner registry and no extra transaction are needed. Delivery goes to the owner's staked wallet; a holder may stamp a payment address into their datum (owner-signed, self-trustworthy) for finer delivery.
- Native mint: exactly $\text{burned} \times v4 / \text{scale}$, regardless of which credential signed the burn — the native policy approves the mint only because it is backed by the governed burn of the same name at the schedule's final value ("no burn CIP, no mint", scaled). $v4$ is the baked constant ($\approx 1.1699 \times \text{scale}$): the only on-chain arithmetic is this final multiply. The mint is deliberately not 1:1 quantity-conserved — the scaled value rules that out — so it is governed by the native policy and the rule script instead.
- Third-party path mechanics: the PLB spend's paired continuation must preserve address, datum and reference script byte-for-byte (lovelace is **ratcheted** — output $\geq$ input, not conserved) — the **ghost** output (one-time per spent UTxO, reclaimable by the owner via a transfer-path spend). In the issuer's case the ghost remains; on the owner path (transfer path + the same Burn mode) nothing remains — the cheaper shape.
- Base-layer guarantee: a transaction that spends a registry node can never mint or burn that node's own token — graduation is pure issuance, never mixed with a registry reconfiguration.
- Asset identity (Q-GRAD-1): the native asset is a new policy — DEX pools / price history continuity across the flip remains an open global question.

Figure 4: Deadline graduation (permissionless, value-preserving)